Як насправді працюють Full Micro, MicroMacro та Structural A/U–E/U–E/E

Просте послідовне пояснення з однією мережею, формулами та схемами

Що треба зрозуміти з цього тексту

Усі варіанти нижче описують поширення інфекції в мережі, але вони **бачать різну кількість інформації про міжспільнотні контакти**. Full Micro бачить кожне ребро. Original MicroMacro бачить детальну мережу всередині кожної community, але між communities працює з агрегатами. Structural A/U, E/U та E/E — це окремий діагностичний simulator, де ми по одному повертаємо інформацію, втраченою агрегацією.

1. Почнемо з однієї й тієї самої мережі

Нехай є дві communities: A і B. Усередині кожної є звичайна контактна мережа людей. Між communities існують лише кілька реальних контактів. Червоні вузли вже заражені, сірі — susceptible. Синя рамка означає boundary/gateway-вузол, який має контакт з іншою community.

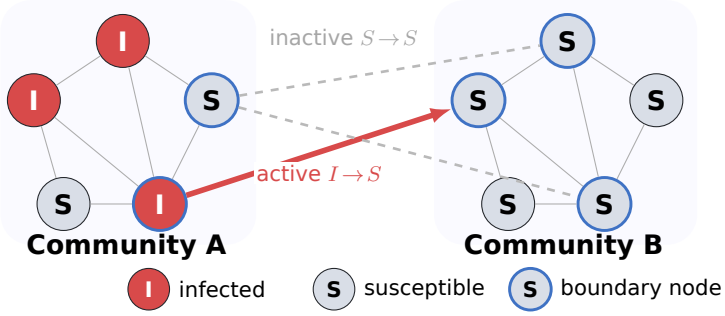


Figure 1: Одна фізична мережа. Між communities є три реальні cross-edges, але зараз активне лише одне $I \to S$ ребро. Саме цю різницю між "усіма ребрами" та "активними ребрами" далі бачать або втрачають різні моделі.

Ця сама мережа буде подана всім симуляторам. Відмінність не в самій картинці, а в тому, **яку частину цієї картинки алгоритм використовує для міжcommunity передачі**.

2. Full Micro: алгоритм бачить усе

Full Micro — це повний Gillespie на всій мережі. Для нього немає принципової різниці між "внутрішнім" і "міжcommunity" ребром. Усі контакти — це звичайні ребра одного графа.

Для кожного ребра (u, v), де u infected, а v susceptible, існує канал зараження з rate

$$a_{uv} = \beta w_{uv} \mathbb{I}[X_u = I,\, X_v = S].$$

Якщо ребра невагові, $w_{uv} = 1$. Загальний infection hazard — це сума rates усіх активних $I \to S$ ребер:

$$\Theta_{\text{micro}} = \sum_{(u,v)\in E} a_{uv}.$$

Тобто Full Micro буквально запитує: *"Які конкретно заражені люди зараз мають контакт із конкретними susceptible людьми?"*

У мережі на рис. 1 між A і B є три фізичні ребра, але активне зараз тільки одне. Якщо всі мають rate β, точний міжcommunity hazard дорівнює

$$\lambda_{AB}^{\text{exact}} = \beta.$$

Коли це ребро спрацьовує, заражається **саме той susceptible-вузол, який стоїть на його кінці**. Full Micro не вигадує target: його визначає реальне ребро.

Інтуїція

Full Micro = "я знаю стан кожної людини, кожне ребро і тому точно знаю, які переходи зараз можливі".

Це відповідає твоєму попередньому exact Gillespie-підходу на contact network: інтенсивність susceptible-вузла визначається його зараженими сусідами, а наступний event вибирається серед реально доступних transitions.

3. Original MicroMacro: детально всередині, грубо між communities

MicroMacro робить головне спрощення: **усередині community він усе ще запускає локальний Gillespie на конкретних вузлах**, але між communities стискає інформацію.

Для macro-рівня community A перетворюється приблизно на такий запис:

$$A: \quad I_A,\ S_A,\ N_A,$$

а пара communities A, B має агреговану вагу W_{AB}, яка описує силу їхнього зв'язку.

У твоїй MoMLeT-моделі міжcommunity rate записується як

$$\lambda_{AB}(t) = \beta\, W_{AB}\, C_A(t)\, S_B(t)\, T,$$

де C_A — агрегована contagiousness source-community, а S_B — агрегована susceptibility destination-community. Для інтуїції це можна читати як схему

$$\lambda_{AB}^{A/U} \propto \beta W_{AB} \frac{I_A}{N_A} \frac{S_B}{N_B}.$$

У нашому toy example $W_{AB} = 3$ (три міжcommunity ребра), $I_A/N_A = 2/5$, а $S_B/N_B = 1$. Тоді агрегована формула може дати hazard близько

$$\lambda_{AB}^{A/U} = 3\beta \cdot \frac{2}{5} = 1.2\beta,$$

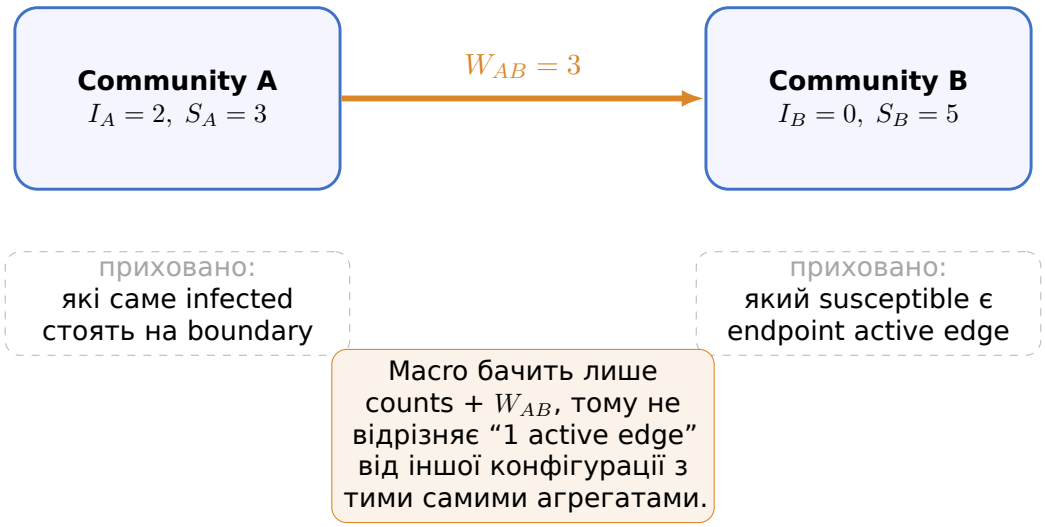


Figure 2: Macro-представлення тієї самої мережі. Локальні вузли не зникають із micro-models, але для міжcommunity hazard production MicroMacro використовує агреговані counts і W_{AB}, а не поточні стани конкретних cross-edges.

хоча Full Micro бачить лише одне реально активне ребро і дає β.

Це і є **структурна апроксимація**: macro-рівень припускає, що infected і susceptible “розмішані” відносно boundary приблизно так само, як у всій community.

Коли macro-event відбувся, original MicroMacro не знає точний endpoint cross-edge. Тому він додає infection у **випадковий susceptible-вузол** destination-community.

3.1. Ще одна різниця: як MicroMacro рухає час

Original MicroMacro не просто має aggregate hazard. Він має власний production scheduler. Micro communities просуваються блоками довжини τ_{micro}, а macro-рівень накопичує hazard

$$H(t) = \int_{t_0}^{t} \Theta(s)\, ds$$

до випадкового порогу $E = -\ln U$. У статті цей інтеграл апроксимується на micro-grid, зокрема через midpoint rate в кожному інтервалі.

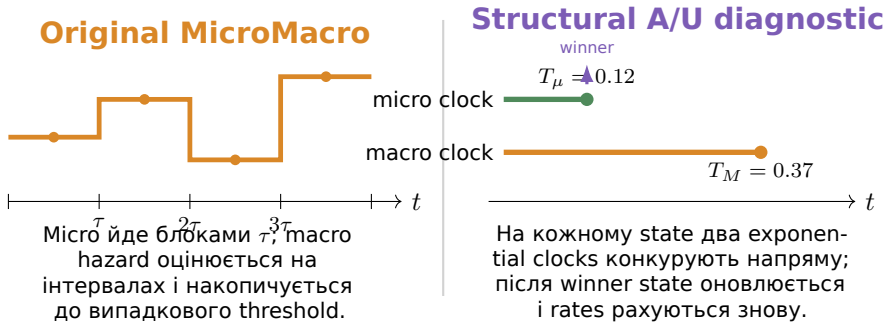


Figure 3: Різниця scheduling. Production MicroMacro використовує τ-синхронізацію та cumulative-hazard integration; A/U diagnostic залишає ту саму aggregate A/U ідею, але використовує direct event competition.

4. Structural A/U: ті самі грубі припущення, але інший simulator

Оце найважливіший момент.

Structural A/U означає:

$$\boxed{A = \text{Aggregate hazard}}, \qquad \boxed{U = \text{Uniform placement}}.$$

Тобто за **структурним змістом** A/U робить те саме грубе припущення, що й original MicroMacro:

- міжcommunity hazard рахується з агрегованої інформації, а не зі стану конкретних cross-edges;
- після import заражається випадковий susceptible у destination-community.

Але A/U — **не той самий executable code**. У diagnostic implementation немає production τ-scheduler у тому самому вигляді. Там micro і macro події конкурують як event-driven stochastic clocks.

Навіщо взагалі потрібен A/U diagnostic?

Від Full Micro original MicroMacro відрізняється одразу у двох місцях. По-перше, він замінює точні міжcommunity active edges на **aggregate hazard** і після import використовує **uniform placement**. По-друге, він використовує власний production τ-scheduler замість прямої глобальної конкуренції подій.
A/U diagnostic залишає першу відмінність, але прибирає другу: **aggregate hazard + uniform placement залишаються, а production τ-scheduler замінено на direct event competition**. Тому порівняння Full Micro з A/U показує переважно структурну похибку самої A/U-апроксимації, без домішки production synchronization mechanics.

Тому різниця така:

	Original MicroMacro	**Structural A/U**
Macro information	aggregate	aggregate
Placement	uniform susceptible	uniform susceptible
Hazard idea	та сама A/U approximation	та сама A/U approximation
Time scheduler	production τ + hazard integral	окремий event-driven scheduler
Executable	original user-facing simulator	окремий diagnostic simulator

Отже відповідь на питання “*чи hazard рахується інакше?*” така:

Коротка відповідь

У A/U **не обов’язково інша structural formula hazard**; за змістом вона така сама aggregate approximation, як у MicroMacro. Головна відмінність A/U від original MicroMacro — **не formula, а simulation scheduler та implementation mechanics**. Саме тому A/U потрібен, щоб подивитися на “чисту” structural approximation без частини production implementation artifacts.

4.1. Що показало порівняння A/U з Full Micro і MicroMacro

Числа нижче — для одного й того самого baseline: 100 paired мереж, $L = 10$, $n = 100$, $d \approx 6$, $k = 5$, $\beta_{\text{in}} = \beta_{\text{out}} = 0.003$, SI, $T_{\text{end}} = 4000$. Основна метрика — RMST до досягнення останньої community: більший RMST означає **повільніше** проходження інфекції вздовж ланцюга.

Що таке RMST і як він тут рахується

RMST — це *Restricted Mean Survival Time*, тобто середній час до події, але з фіксованим максимальним горизонтом спостереження. У цьому експерименті "подія" — **перше досягнення інфекцією останньої community** в ланцюгу. Для replication r позначимо цей час як $T_{\text{last}}^{(r)}$. Якщо остання community досягнута до кінця симуляції, беремо фактичний час. Якщо ні, replication не викидається: її внесок обмежується горизонтом $T_{\text{end}} = 4000$.

Для R replications:

$$\text{RMST} = \frac{1}{R}\sum_{r=1}^{R} \min\left(T_{\text{last}}^{(r)}, T_{\text{end}}\right).$$

Наприклад, якщо три runs дали terminal arrival 2800, 3200 і "не досягнуто до 4000", то в середнє входять 2800, 3200 і 4000:

$$\text{RMST} = \frac{2800 + 3200 + 4000}{3} = 3333.3.$$

Якщо всі runs завершилися до горизонту, RMST тут просто збігається зі звичайним середнім T_{last}. **Більший RMST** означає пізніше проходження інфекції до кінця chain, тобто повільнішу propagation. У таблицях

$$\Delta\text{RMST} = \text{RMST}_{\text{variant}} - \text{RMST}_{\text{Full Micro}}.$$

Тому від'ємне Δ означає, що variant доходить до останньої community раніше за Full Micro, а додатне — пізніше.

Simulator	**RMST**	**Δ vs Full Micro**
Full Micro	2936.73	0
Structural A/U	2715.94	-220.79
Original MicroMacro, current Production ($\tau = 1$)	1677.55	-1259.18
Production + independent RNG streams	2729.43	-207.30

A/U приходить до кінця в середньому приблизно на **221 одиницю часу раніше**, ніж Full Micro ($\Delta\text{RMST} = -220.8$; 95% CI приблизно $[-315.5, -126.8]$). Це і є чистий сигнал, що навіть без production scheduler aggregate hazard + uniform placement уже дають систематичне прискорення.

Водночас current Production MicroMacro був ще набагато швидшим: RMST 1677.55. Різниця між ним і event-driven A/U становила близько **1038 RMST**. Але цей великий gap **не можна приписувати самому τ-scheduler**. Після виправлення shared-RNG architecture Production дав RMST 2729.43 і відрізнявся від A/U лише на $+13.49$ RMST; 95% CI цього contrast включає нуль. Окремо зменшення τ від 1 до 0.25 змінювало RMST лише приблизно на 31.15.

Що означають “current Production” і “Production + independent RNG streams”

Це **не дві різні наукові моделі**. Обидва варіанти використовують той самий production MicroMacro: ті самі community MicroModels, aggregate macro hazard, uniform import placement, $\tau = 1$, midpoint/cumulative-hazard scheduler і normal repeated imports. У variant “Production + independent RNG streams” змінено лише те, **звідки кожна micro-community бере випадкові числа**.

У current Production кожен `MicroModel` створюється без власного RNG. Тоді всі communities використовують один і той самий module-level Python `random`, тобто **один спільний RNG state**. Сам по собі один спільний pseudorandom generator ще не був би помилкою. Проблема виникає через комбінацію **shared RNG + horizon-safe rollback**.

Коли `simulate_until(t_end)` пробує наступну Gillespie event і бачить, що її час лежить за поточним horizon, він повертає RNG у стан `rng_before`. Для окремого RNG це локальна операція. Але в current Production цей state є глобальним для всіх communities. Тому rollback однієї community повертає назад random stream також для наступної community, яка симулюється після неї.

Чому з’являється штучна залежність

Уявімо, що перед симуляцією community A глобальний RNG має стан R. A генерує random draw, але event лежить за horizon, тому код відновлює state назад до R. Потім симулюється community B. Вона стартує з того самого R і може повторно використати ту саму послідовність random draws. Отже stochastic evolution B починає залежати від того, як A просувалася до horizon і чи зробила rollback. Це **cross-community RNG coupling та order dependence**, а не фізичний зв’язок між communities.

У correction variant кожній micro-community явно призначається власний generator,

$$\mathrm{RNG}_1, \mathrm{RNG}_2, \ldots, \mathrm{RNG}_L,$$

з окремим deterministic seed. Тоді rollback community i змінює тільки state RNG_i і не може повернути random stream іншої community. Midpoint clones також копіюють/відновлюють саме per-community RNG state. Macro budget і pair selection при цьому залишаються частиною original production machinery; scientific aggregate A/U semantics та τ-scheduler не змінюються.

Окремий unit diagnostic підтвердив, що це не лише теоретична можливість. Для двох однакових ізольованих micro-processes shared-global mode дав correlation фінальних infected counts приблизно 0.531; для independent streams — -0.002. Ще сильніший сигнал: імовірність заразити target у першої community була 0.394, а у другої лише 0.155 при shared RNG. Після розділення streams значення стали приблизно 0.394 і 0.390. Тобто shared RNG + rollback змінював навіть marginal distribution другої community.

Implementation	Micro scheduling / state	Cross-community model	RNG architecture
Full Micro	один global Gillespie process на full graph	exact active edges, exact endpoint	один RNG для одного global process
Current Production MM	окремі MicroModels + τ-synchronization	aggregate hazard + uniform import	один shared global RNG між micro communities
Production + independent RNG	**той самий production scheduler**	**та сама aggregate/uniform model**	окремий RNG для кожної micro-community
Structural A/U	один diagnostic global node state, event-driven race	aggregate hazard + uniform placement	окремі streams за stochastic roles; не production scheduler
E/U, E/E	той самий diagnostic event-driven framework	exact hazard; E/U uniform, E/E exact placement	diagnostic streams, не production per-community architecture

Тому назва “Production + independent RNG streams” означає саме **мінімальну implementation correction original MicroMacro**, а не новий E/U/E/E simulator. Її призначення — перевірити, що станеться, якщо зберегти production algorithm, але прибрати штучне stochastic coupling між локальними micro-simulators.

Як це читати

Full Micro $\to$ **A/U** показує головним чином structural A/U error.
A/U $\to$ **current Production MicroMacro** у цій версії показував переважно implementation/RNG problem, а не “похибку від τ” як таку. Після RNG correction production і A/U практично збіглися.

5. E/U: виправляємо тільки hazard

Тепер ми залишаємо uniform placement, але повертаємо точну інформацію про реальні міжcommunity ребра.

Для pair $A \to B$ exact hazard:

$$\lambda_{AB}^{E/U} = \beta \sum_{(u,v)\in E_{AB}} w_{uv}\, \mathbb{I}[X_u = I,\, X_v = S].$$

У toy network з рис. 1 активне тільки одне cross-edge, тому

$$\lambda_{AB}^{E/U} = \beta,$$

а не 1.2β, як у нашому простому aggregate прикладі.

Але після того, як macro-event відбувся, E/U все ще робить “U”: вибирає **будь-який susceptible** у B, а не обов’язково endpoint активного ребра.

Тому перехід A/U $\to$ E/U вимірює насамперед помилку від **aggregate hazard**. Обидва варіанти запускаються тим самим diagnostic event-driven scheduler і в обох placement залишається uniform. Змінюється лише джерело macro hazard: у A/U він агрегований, а в E/U рахується з реальних active $I \to S$ cross-edges.

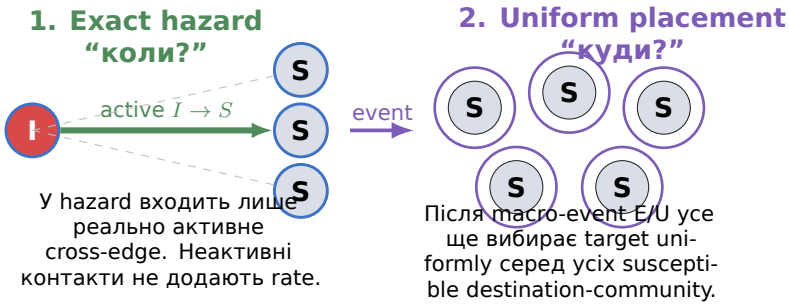


Figure 4: E/U виправляє тільки hazard. Він точно визначає інтенсивність міжcommunity передачі з active edges, але placement після event залишається uniform.

Що саме перевіряє E/U?

Якщо E/U стає помітно ближчим до Full Micro, ніж A/U, це означає, що значна частина structural error виникала через **втрату інформації про те, які міжcommunity ребра реально активні**. Scheduler тут не міняється, тому його вплив не плутається з ефектом exact hazard.

5.1. Що показало порівняння E/U з Full Micro і MicroMacro

На тому самому baseline E/U дав:

Simulator	RMST	Δ vs Full Micro
Full Micro	2936.73	0
Structural E/U	2775.5	-161.2
Structural A/U	2715.94	-220.79
Original MicroMacro, current Production	1677.55	-1259.18

Після заміни aggregate hazard на exact active-edge hazard RMST зріс приблизно на **59.6** відносно A/U. Тобто E/U рухається в правильний бік — ближче до Full Micro. Але від Full Micro все ще залишається значущий gap: приблизно -161.2 RMST, 95% CI $[-266.8, -48.8]$.

Причина проста: E/U уже правильно знає *коли* міжcommunity infection може відбутися, але після event усе ще може покласти infection у неправильне місце, бо target залишається uniform.

Порівняно з current Production MicroMacro E/U має RMST приблизно на **1098** більший. Цей raw gap не слід трактувати як “ефект exact hazard”: current Production baseline містив великий shared-RNG implementation defect. Інформативніше дивитися на послідовність A/U $\to$ E/U, де scheduler однаковий і змінюється тільки hazard.

Головний результат E/U

Exact hazard **частково** повертає поведінку до Full Micro, але не закриває весь structural gap. Отже знати лише правильну сумарну інтенсивність недостатньо: важливо ще й правильно визначити, *який* вузол у destination-community отримує import.

6. E/E: виправляємо і hazard, і placement

E/E означає:

$$\boxed{E = \text{Exact hazard}}, \qquad \boxed{E = \text{Exact placement}}.$$

Hazard такий самий exact active-edge sum, як в E/U. Але тепер, якщо event вибрано через конкретне ребро (u, v), заражається саме його endpoint v.

Тобто E/E знає обидві речі:

1. **як часто** має відбуватися inter-community transmission;
2. **куди саме** він може потрапити.

Перехід E/U $\rightarrow$ E/E ізолює помилку від **uniform placement**. Scheduler і exact hazard залишаються такими самими, як в E/U; змінюється лише правило вибору target-вузла після міжcommunity event.

Що саме перевіряє E/E?

E/E відповідає на питання: "якщо ми вже правильно знаємо *коли* має відбутися міжcommunity зараження, скільки ще похибки дає те, що Micro-Macro не знає *який саме* вузол має заразитися?" Якщо E/E майже збігається з Full Micro, це також є sanity check, що diagnostic event-driven implementation правильно відтворює exact process, коли йому повернули всю потрібну edge-level інформацію.

6.1. Що показало порівняння E/E з Full Micro і MicroMacro

Результат E/E був майже таким самим, як Full Micro:

Simulator	RMST	Δ vs Full Micro
Full Micro	2936.73	0
Structural E/E	2944.3	$+7.5$
Structural E/U	2775.5	-161.2
Original MicroMacro, current Production	1677.55	-1259.18

Різниця E/E vs Full Micro становила лише $+7.5$ RMST, а 95% CI $[-93.6, +120.3]$ включав нуль. Тобто в цьому baseline немає статистичних підстав вважати, що E/E систематично відрізняється від Full Micro.

Перехід E/U $\rightarrow$ E/E дав приблизно **+168.8 RMST** (тобто propagation стала пізнішою і ближчою до Full Micro); відповідний 95% CI для improvement приблизно $[56.1, 274.5]$. Це показує, що при вже правильному hazard саме placement був великим залишковим джерелом structural error.

Порівняно з current Production MicroMacro E/E має RMST приблизно на **1267** більший. Тут це очікувано: E/E уже повертає exact edge-level information і тому поводиться як Full Micro, тоді як current Production мав одночасно structural approximation та implementation defect.

Головний результат E/E

У цьому baseline E/E практично відтворює Full Micro. Це сильний sanity check: коли diagnostic simulator отримує exact hazard і exact target, різниця з глобальним Full Micro зникає в межах Monte-Carlo uncertainty.

7. То яка різниця між E/E і Full Micro?

Після перевірки реалізацій тут можна сказати точніше: **у Experiment 01 E/E і Full Micro налаштовані так, щоб задавати той самий CTMC generator**. Вони відрізняються не правилами зараження, а способом, яким одна й та сама множина подій організована та вибирається в коді.

Full Micro отримує весь `full_graph` і передає його одному `MicroEngine`. Для нього внутрішнє ребро і cross-community edge — просто два ребра одного графа. Кожне активне ребро $u(I) - v(S)$ дає infection channel

$$a_{uv} = \beta w_{uv},$$

а recovery кожного infected-вузла має rate γ. Усі infection та recovery channels входять в одну Gillespie competition.

E/E також працює з одним глобальним станом усіх вузлів. Важливо: у ньому немає окремого "micro A simulator" і "micro B simulator". Код лише **індексує ребра двома групами**: internal edges та cross-community edges. Для internal активного $I-S$ ребра використовується $\beta_{\text{in}} w$; для cross-community активного $I - S$ ребра — $\beta_{\text{out}} w$.

У runner цього експерименту є явна перевірка

$$\beta_{\text{in}} = \beta_{\text{out}},$$

оскільки Full Micro має один infection rate для всіх ребер. Тому в Experiment 01 відповідне активне ребро має **той самий rate** у Full Micro та E/E. Recovery rule також однаковий: кожний infected-вузол має rate γ.

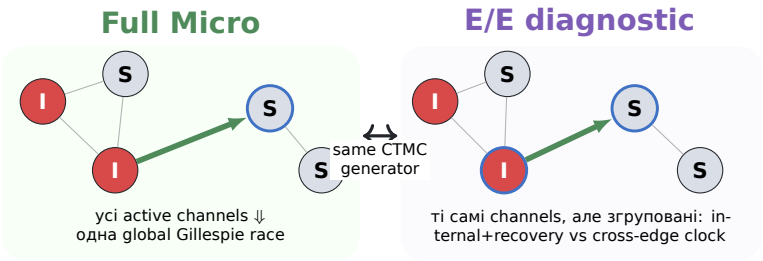


Figure 5: У Experiment 01 E/E не має окремих micro-simulators для communities. Він має один глобальний node state, але програмно групує event channels на internal/recovery та cross-community. Full Micro тримає всі ці channels в одній Gillespie race.

Чому дві групи clocks не змінюють процес? Нехай

$$\Lambda_\mu = \Lambda_{\text{internal}} + \Lambda_{\text{recovery}}, \qquad \Lambda_M = \Lambda_{\text{cross}}.$$

Full Micro генерує один час

$$\Delta t \sim \text{Exp}(\Lambda_\mu + \Lambda_M).$$

E/E генерує два незалежні часи

$$T_\mu \sim \text{Exp}(\Lambda_\mu), \qquad T_M \sim \text{Exp}(\Lambda_M)$$

і бере найменший. Для exponential clocks

$$\min(T_\mu, T_M) \sim \mathsf{Exp}(\Lambda_\mu + \Lambda_M),$$

а ймовірність перемоги кожної групи пропорційна її rate. Тому така дворівнева організація sampling математично еквівалентна одній глобальній Gillespie race.

Для cross-community event E/E також зберігає точні ймовірності конкретних активних ребер. Спочатку pair $i \to j$ вибирається пропорційно сумі rates його активних cross-edges. Потім target вибирається пропорційно сумі ваг активних ребер, що входять у нього, а source — пропорційно вазі відповідного active edge. У результаті конкретне ребро (u, v) вибирається з тією самою ймовірністю, що й у Full Micro.

Отже, тут не треба писати “if rules match”

Для конфігурації Experiment 01 умови еквівалентності **перевірені самою реалізацією**: той самий full graph та initial node, $\beta_{\mathsf{in}} = \beta_{\mathsf{out}}$, той самий γ, ті самі SIR event rules, exact active cross-edge hazard і exact edge-conditioned placement. Тому E/E та Full Micro повинні мати один статистичний розподіл, хоча конкретні trajectories різняться через іншу організацію RNG streams.

Саме це перевіряє oracle gate. У 100 paired replications Full Micro мав RMST 2936.7, E/E — 2944.3; різниця $+7.5$ з 95% CI $[-93.6, +120.3]$. Інтервал містить нуль, тобто дані не показують систематичної різниці між двома реалізаціями. Це узгоджується з перевіркою коду: E/E є альтернативною реалізацією того самого CTMC, а не ще однією approximation.

Перевірено за реалізаціями: `simulators/micro.py`, `engines/micro_engine.py`, `hazard_placement_ablation/simulator.py` та `hazard_placement_ablation/runner.py`.

8. Уся логіка A/U → E/U → E/E одним ланцюжком

Перед самим ablation ladder корисно тримати в голові дві різні причини, чому original MicroMacro може відрізнятися від Full Micro:

Structural: exact boundary edges → aggregate community hazard.

Scheduler: global event competition → τ-synchronized hazard integration.

A/U diagnostic спеціально прибирає другу різницю, але залишає першу. Далі A/U → E/U повертає exact hazard, а E/U → E/E повертає exact placement.

Variant	RMST	Δ vs Full Micro	Δ vs current MM
Current Production MicroMacro	1677.55	-1259.18	0
A/U	2715.94	-220.79	$+1038.39$
E/U	2775.5	-161.2	$+1098.0$
E/E	2944.3	$+7.5$	$+1266.8$
Full Micro	2936.73	0	$+1259.18$

Ця таблиця добре показує дві різні історії. Великий стрибок від current Production до A/U був переважно implementation/RNG correction, тоді як подальші кроки A/U → E/U → E/E уже послідовно повертають structural information і ведуть diagnostic model до Full Micro.

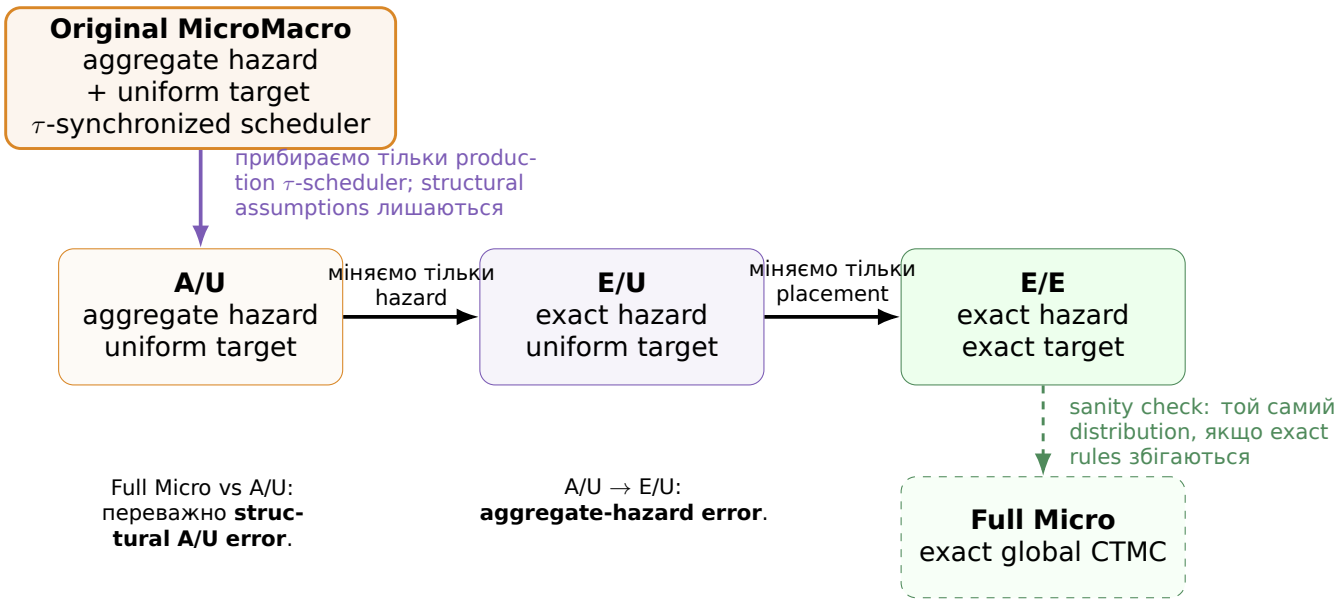


Figure 6: Що саме ізолює кожен крок. A/U відділяє structural assumptions від production scheduling; E/U змінює лише hazard; E/E змінює лише placement і служить sanity check проти Full Micro.

9. Що вважати основним результатом для статті

Тут важливе методологічне правило.

Якщо ми хочемо сказати щось саме про **твій user-facing MicroMacro method**, primary experiments краще робити на original ‘MicroMacroSimulator‘ і змінювати мінімально необхідну деталь:

Experiment	**Найчистіша реалізація**
A/U baseline	original MicroMacro без змін
E/U	original MicroMacro + замінити лише macro hazard provider на exact active-edge hazard
E/E	те саме + замінити лише target selector на exact edge endpoint
Gateway near/random/far	simulator взагалі не змінювати; змінювати тільки topology
$d \times k$ grid	simulator не змінювати; змінювати лише параметри мережі

Окремий ‘HazardPlacementSimulator‘ все одно корисний: він дуже добре підходить як **secondary mechanistic diagnostic**, бо прибирає production scheduler artifacts і дозволяє чисто розкласти structural error. Але його результати краще подавати як підтвердження механізму, а не як єдине свідчення того, що отримає користувач original MicroMacro.

10. Найкоротше резюме

Full Micro: бачить кожен вузол і кожне active edge. Exact hazard, exact target.

Original MicroMacro: усередині community бачить nodes, між communities бачить aggregate I, S, W. Aggregate hazard, uniform target, production τ-scheduler.

Structural A/U: ті самі aggregate/uniform structural assumptions, але окремий event-driven diagnostic simulator; тому він прибирає production τ-scheduler і залишає structural A/U approximation.

E/U: той самий diagnostic scheduler і той самий uniform placement, але aggregate hazard замінено на exact cross-edge hazard; різниця A/U $\rightarrow$ E/U ізолює hazard information loss.

E/E: той самий diagnostic scheduler та exact hazard, але uniform placement замінено на exact endpoint; різниця E/U $\rightarrow$ E/E ізолює placement error. Якщо решта event rules збігаються з Full Micro, статистично має відтворювати той самий CTMC.

Примітка про джерела

Опис original MicroMacro, його двох рівнів, macro-rate та cumulative-hazard synchronization базується на наданій роботі *Towards complexity reduction of large-scale epidemic simulation in two-scale networks* (MoMLeT-2025). Опис Full Micro спирається на надану роботу *Simulating epidemic peak dynamics on complex networks using efficient Gillespie algorithms*. Розрізнення Structural A/U–E/U–E/E як окремої diagnostic implementation базується на перевіреному source code експериментів у Google Drive в межах поточного аудиту. Числові результати RMST взято з Experiment 01 (factorial hazard/placement ablation, 31.07.2026, 100 paired replications) та Experiment 07 (production implementation decomposition, аналіз 14.08.2026).